PYTHON FUNDAMENTALS

Detailed General Study Notes

Explanations • examples • common pitfalls • review questions

Designed as regular educational material for learners and for testing document-based tutoring systems. No application-specific IDs or schema fields are embedded in the study chapters.

Contents

1. Getting Started: Syntax, Variables, and Data Types

2. Operators, Expressions, and Console Input/Output

3. Conditional Statements and Decision Making

4. Loops and Iteration

5. Working with Strings

6. Lists, Tuples, and Sets

7. Dictionaries and Key-Value Data

8. Functions, Parameters, and Variable Scope

9. List and Dictionary Comprehensions

10. Modules, Imports, and Code Organization

11. Exception Handling and Errors

12. Reading and Writing Files

13. Recursion and Recursive Problem Solving

How to use these notes

Read a section, run its example in Python 3, and answer the review questions without checking the explanation first. The examples are illustrative; input-based programs require learner input when run. The chapters are organized for a beginner-to-intermediate overview.

1. Getting Started: Syntax, Variables, and Data Types

How Python programs work

Python is a general-purpose language whose statements are usually executed from top to bottom. Indentation defines code blocks; it is not merely visual formatting. Comments begin with # and are ignored by Python. Names are case-sensitive, so total and Total are different variables.

Variables and assignment

A variable refers to a value. The assignment operator = binds a name to a value; it is different from ==, which compares values. Names should be descriptive, start with a letter or underscore, and cannot be Python keywords. Values have types, while variables can later refer to values of different types.

Common types

int represents whole numbers; float represents floating-point numbers; str holds Unicode text; bool holds True or False; and None represents the absence of a value. Use type(value) to inspect a type. A string containing digits is still a string until converted.

Type conversion

Use int(), float(), str(), and bool() where appropriate. Conversion can fail: int("hello") raises ValueError. Decimal-looking strings such as "3.5" are typically converted with float() rather than int().

Worked Python example

```
name = "Anu"
age = 19
height = 1.65
is_student = True
print(name, age, type(height))
next_year = age + 1
number = int("42")
print(number + 8)
```

Check your understanding

1. Explain why x = 5 and x == 5 mean different things.
2. What happens if you run int("three")?

2. Operators, Expressions, and Console Input/Output

Arithmetic expressions

Python supports +, -, *, /, //, %, and **. / returns a floating-point quotient; // performs floor division, which rounds toward negative infinity. ** computes powers. Parentheses improve clarity when expressions mix operators.

Comparisons and logic

The comparisons ==, !=, <, <=, >, >= produce Boolean results. Combine conditions with and, or, and not. Expressions may short-circuit: in A and B, B is not evaluated when A is false.

Input and output

print() displays values. input() reads a line as a string even if it looks numeric. Convert input explicitly before arithmetic, and validate it when invalid input is possible. Formatted strings, or f-strings, embed expressions inside braces.

Worked Python example

```
price = 120
quantity = 3
print(f"Total: {price * quantity}")
print(7 / 2, 7 // 2, 7 % 2)
age_text = input("Enter your age: ")
age = int(age_text)
print(age >= 18 and age < 65)
```

Check your understanding

1. Why does input() return a string?
2. What is the difference between / and //?

3. Conditional Statements and Decision Making

Branching

Use if to test a condition, elif to try another condition, and else for the remaining case. Exactly one branch in a single if/elif/else chain is chosen. Indentation identifies statements belonging to each branch.

Truth values

Zero, empty strings, empty collections, and None are false-like values. Non-empty strings are true-like, including "False". Prefer explicit comparisons when they communicate intent more clearly.

Nested decisions

An if statement can appear inside another block. Deep nesting can make code harder to read, so combine logical conditions or move reusable decisions into functions where useful.

Worked Python example

```
marks = 76
if marks >= 90:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Needs improvement"
print(grade)
```

Check your understanding

1. Why is indentation important?
2. When would elif be preferable to a separate if?

4. Loops and Iteration

Repeating tasks

A for loop iterates over items in an iterable such as a list, string, or range object. A while loop repeats as long as its condition remains true. Ensure a while-loop condition can eventually become false unless an intentional infinite loop is required.

range()

range(stop) creates integers from zero up to, but not including, stop. range(start, stop, step) supports custom starts and increments, including negative steps.

Loop controls

break exits the nearest enclosing loop immediately. continue skips to the next iteration. An optional loop else block runs if the loop finishes without hitting break.

Worked Python example

```
for number in range(1, 6):
    if number == 3:
        continue
    print(number)

countdown = 3
while countdown > 0:
    print(countdown)
    countdown -= 1
```

Check your understanding

1. What does range(5) produce?
2. How do break and continue differ?

5. Working with Strings

Text as a sequence

A string is an immutable sequence of Unicode characters. Indexing begins at zero; negative indices count from the end. Slicing text[start:stop:step] includes start but excludes stop.

Common methods

strip() removes leading and trailing whitespace, lower() and upper() change case, replace() substitutes text, split() separates a string into pieces, and join() combines a sequence of strings. These operations return new strings.

Formatting and validation

Use f-strings for readable formatting. Check membership with in. Methods such as isdigit() and startswith() are useful for simple checks, but thorough validation may require additional rules.

Worked Python example

```
message = "  Hello, Python  "
print(message.strip().lower())
print(message[2:7])
items = "red,green,blue".split(",")
print(" | ".join(items))
username = "sam"
print(f"Welcome, {username}!")
```

Check your understanding

1. Why does replacing part of a string create a new value?
2. What is the output of "python"[::-1]?

6. Lists, Tuples, and Sets

Lists

A list is an ordered, mutable collection that can contain different types. Access items by index or slice; append() adds one item, extend() adds many, and pop() removes an item. Sorting a list with sort() changes the original list.

Tuples

A tuple is an ordered, immutable collection, often used for a fixed record. Parentheses are common, but the comma makes a single-item tuple: (5,).

Sets

A set stores unique hashable elements and supports union, intersection, and difference. Sets do not provide positional indexing, and their iteration order should not be treated as a fixed sequence.

Choosing a collection

Use a list for an editable sequence, a tuple for a fixed record, and a set for membership checks or removing duplicates. The correct choice depends on the operations needed.

Worked Python example

```
scores = [80, 90, 80]
scores.append(95)
print(scores[0], len(scores))
location = ("Kochi", 10.0)
unique_scores = set(scores)
print(unique_scores)
print({1, 2, 3} & {2, 3, 4})
```

Check your understanding

1. How is a tuple different from a list?
2. Why can a set not be accessed with set_value[0]?

7. Dictionaries and Key-Value Data

Dictionary structure

A dictionary maps unique hashable keys to values. Keys often are strings or numbers; values can be any Python objects, including other dictionaries. Dictionaries preserve insertion order in modern Python.

Reading and changing entries

Use data[key] when the key must exist; a missing key raises KeyError. Use get(key, default) when missing keys are expected. Assign data[key] = value to add or update an entry; use pop() to remove one.

Iterating and nesting

Use items() to loop over key-value pairs. Nested dictionaries represent structured records, but missing inner keys should be handled carefully.

Worked Python example

```
student = {"name": "Anu", "marks": 86}
print(student["name"])
print(student.get("email", "not provided"))
student["marks"] = 91
for key, value in student.items():
    print(key, value)
```

Check your understanding

1. How does get() differ from square-bracket lookup?
2. Can a list be used as a dictionary key? Why not?

8. Functions, Parameters, and Variable Scope

Reusable operations

A function groups statements into a named, reusable unit. Define one with def; call it using its name and parentheses. Parameters receive arguments when a function is called.

Return values

return sends a result back to the caller and ends execution of that function. Without an explicit return, a function returns None. Printing a value is not the same as returning it.

Scope

Names assigned inside a function usually belong to local scope. Global names can be read from inside functions, but changing them requires care. Passing inputs and returning outputs is generally easier to test than relying on mutable global state.

Defaults and arguments

Functions may use positional arguments, keyword arguments, and default values. Avoid mutable default values such as [] when each call should receive fresh data.

Worked Python example

```
def calculate_total(price, quantity=1):
    subtotal = price * quantity
    return subtotal

print(calculate_total(50, 3))
print(calculate_total(price=25))

def greet(name):
    print(f"Hello, {name}")
```

Check your understanding

1. What is the difference between print and return?
2. What happens to a local variable after a function finishes?

9. List and Dictionary Comprehensions

Concise collection building

A comprehension creates a collection from an iterable. List comprehensions use [expression for item in iterable]; dictionary comprehensions use {key: value for item in iterable}.

Filtering

Append if condition after the iterable to keep only matching items. A conditional expression inside the result expression can choose one of two values for every item.

Readability

Comprehensions are most useful for short transformations. If the expression becomes difficult to understand or needs multiple side effects, an ordinary loop may be clearer.

Worked Python example

```
numbers = [1, 2, 3, 4, 5]
squares = [n * n for n in numbers]
even_squares = [n * n for n in numbers if n % 2 == 0]
lookup = {n: n * n for n in numbers}
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(even_squares, lookup)
```

Check your understanding

1. How does filtering differ from an if/else expression?
2. When would a normal loop be more readable?

10. Modules, Imports, and Code Organization

Modules

A module is a Python file that can define functions, classes, and variables. Using modules helps organize a program into smaller pieces.

Import styles

import math makes math functions available through the module name, such as math.sqrt(). from math import sqrt brings the specific name into the current namespace. Avoid wildcard imports because they make origins of names unclear.

Standard library

Python includes modules for math, dates, file paths, randomness, and many other tasks. Third-party packages must be installed separately and imported into the chosen Python environment.

Entry-point guard

if __name__ == "__main__": executes a block only when that file is run directly, not when it is imported by another module.

Worked Python example

```
import math
from pathlib import Path

print(math.sqrt(81))
print(Path("notes.txt").suffix)

def main():
    print("This file was run directly")

if __name__ == "__main__":
    main()
```

Check your understanding

1. Why might import math be clearer than importing *?
2. What does the __main__ guard prevent when a module is imported?

11. Exception Handling and Errors

Types of errors

Syntax errors prevent code from being parsed. Runtime exceptions occur while a program runs, for example ZeroDivisionError, ValueError, and FileNotFoundError.

try and except

Place code that might fail inside try. Handle known errors using except followed by the specific exception type. Catching every exception without distinguishing errors may hide bugs.

else, finally, and raise

An optional else block runs when no exception occurs in try. A finally block normally runs whether an exception occurs or not, making it useful for cleanup. raise explicitly triggers an exception when program rules are violated.

Worked Python example

```
text = "12"
try:
    value = int(text)
    result = 100 / value
except ValueError:
    print("Enter a whole number")
except ZeroDivisionError:
    print("The number cannot be zero")
else:
    print(result)
finally:
    print("Calculation finished")
```

Check your understanding

1. Why is except ValueError more precise than except Exception?
2. When does finally run?

12. Reading and Writing Files

Why files matter

Files allow programs to save or load information beyond one execution. Text files store strings; binary files store bytes. Use an appropriate encoding, commonly UTF-8, for text.

Context managers

with open(path, mode, encoding="utf-8") as file: closes the file when the block ends, including after many exceptions. Use "r" to read, "w" to overwrite or create, and "a" to append.

Paths and errors

Use pathlib.Path to work with paths more portably. Opening a missing file in read mode raises FileNotFoundError; programs should either handle it or explain the requirement.

Worked Python example

```
from pathlib import Path

path = Path("example_note.txt")
with path.open("w", encoding="utf-8") as file:
    file.write("Python file handling\n")

with path.open("r", encoding="utf-8") as file:
    content = file.read()
print(content)
```

Check your understanding

1. What is the difference between w and a modes?
2. Why is with recommended for files?

13. Recursion and Recursive Problem Solving

Self-referential solutions

A recursive function calls itself to solve a smaller version of the same problem. Each invocation gets its own local execution context.

Base and recursive cases

A correct recursive solution needs a base case that stops further calls and a recursive step that moves closer to the base case. Without progress, the program may reach its recursion limit.

Trade-offs

Recursion can be expressive for trees, nested structures, and divide-and-conquer problems. Iteration may be simpler or use less call-stack memory for tasks such as calculating factorials.

Worked Python example

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120
```

Check your understanding

1. Identify the base case and recursive step in factorial().
2. What happens if recursion never reaches a base case?